

TCP SYN-Flood Attack and Defense

A Controlled Denial-of-Service Experiment and Two Countermeasures

Course: CSE 406 — Computer Security Sessional

Department of Computer Science and Engineering, BUET

Group members: *[Iftekhar Mahi (2105003)]*, *[teammate 2]*, *[teammate 3]*

September 27, 2026

Abstract

This report documents a controlled TCP SYN-flood (half-open connection) attack carried out entirely between our own devices on an isolated wireless hotspot network, and the countermeasure we validated against it. We describe the attack tooling, the exact steps performed, the measurements collected on the victim (server) and on a separate legitimate-user probe, and we judge whether the attack succeeded and why. We then present the primary real-world defense — stateless SYN cookies (`net.ipv4.tcp_syncookies`, with our own reference implementation in `server/syn_cookies.py`) — and show that it restores service without the collateral damage of naive per-source blocking. A second, application-level countermeasure (`server/defense.py`) is designed and analyzed, but its empirical evaluation was not completed in time for this report; §6.2 states this explicitly rather than presenting unverified numbers.

Contents

1	Objective and Ethical Scope	2
2	Experimental Setup	2
3	(a) Steps of the Attack, with Snapshots	3
4	(c) Observed Output on Each Machine	4
4.1	Victim PC (server)	4
4.2	Monitor PC (the “real user” probe)	5
5	(b) Was the Attack Successful?	6
6	(d) Countermeasures	7
6.1	Countermeasure — stateless SYN cookies (evaluated)	7
6.2	Countermeasure — <code>server/defense.py</code> (SYN Guard): design only, evaluation incomplete	7
6.3	Comparison	8
7	Results Summary	8
8	Conclusion	9
A	Reproduction Commands	9

1 Objective and Ethical Scope

The goal was to reproduce, observe, and defend against a TCP SYN-flood denial-of-service attack. A SYN flood exploits the TCP three-way handshake: the attacker sends a stream of SYN packets, usually with spoofed source addresses, and never completes the handshake. Each such packet forces the victim to allocate a half-open (SYN-RECEIVED) connection and reply with a SYN-ACK. When these half-open entries fill the listen backlog, the server can no longer accept connections from legitimate clients.

Ethical scope. All traffic was confined to hardware we own on a private hotspot with no other devices attached. Source addresses were spoofed only within an unused slice of our own subnet, and the attack was never left running unattended. No third-party system was targeted at any point.

2 Experimental Setup

The experiment used three physical laptops on a single NetworkManager shared Wi-Fi hotspot network (10.42.0.0/24): a *server* (the victim), an *attacker*, and a separate *monitor* laptop that plays the role of an ordinary user and is not involved in generating attack traffic.

Table 1: Roles and software.

Role	Address	Runs
Server / victim	10.42.0.81	nginx (port 80), <code>server/monitor.py</code>
Attacker	(separate laptop)	<code>client/generator.py</code> (raw sockets, root)
Monitor / “real user”	(separate laptop)	<code>client/sustained_http_load.py</code> probe

Attack tool. `client/generator.py` builds complete 40-byte IPv4/TCP SYN packets by hand (custom IPv4 and TCP headers with correct Internet checksums) and sends them over a raw socket. By design, it cycles every non-privileged TCP source port (1024–65535) for one synthetic source IP drawn from a configured pool (10.42.0.192/26) before advancing to the next, so that each half-open connection gets a distinct identity. The packet capture in Figure 1 was taken a few seconds into one flood and shows the arriving stream of SYN packets as recorded at the victim; within that short excerpt the visible source address is 10.42.0.1, the attacker’s own hotspot address, rather than a spoofed one — consistent with the generator still being early in cycling through one source identity’s 64511 ports when the capture was taken.

Measurement tools.

- `server/monitor.py` — samples the victim every 0.5 s into a CSV: `nginx_active`, `port_80_listening`, `syn_recv_count`, `established_count`, `load_1m`, `mem_available_kib`.
- `client/sustained_http_load.py` — a legitimate-user probe, run from the monitor laptop, that repeatedly fetches `http://10.42.0.81/` and records per-request success, latency, and HTTP status.
- `ss` and `tcpdump` for live TCP-state and packet-level evidence on the victim.

Shrinking the queue. A first pass at this experiment showed no effect at all: nginx’s default listen backlog (511 connections) combined with sub-millisecond same-LAN handshake times meant real requests always found a free slot, flood or no flood. Lowering `net.ipv4.tcp_max_syn_backlog` did not help either — that `sysctl` only takes effect for sockets created *after* it is changed, and only

up to whatever value the application itself requests; on this setup nginx's own configured backlog remained the effective limit regardless of the `sysctl`. The only change that actually worked was editing nginx's site configuration directly (`listen 80 ...backlog=8;`) and *restarting* (not reloading) nginx, confirmed afterwards with `ss -ltn`, whose `Send-Q` column for a listening socket reports the backlog actually in effect. All results in this report were collected with this small backlog in place.

3 (a) Steps of the Attack, with Snapshots

The attack and its measurement were performed in the following order.

Step 0 — Reachability and isolation check

From the attacker we confirmed the victim was reachable and that the access point did not isolate clients:

```
ping -c3 10.42.0.81
```

Step 1 — Baseline (no attack)

On the server we started the health sampler; from the separate monitor laptop we ran the legitimate-user probe to record normal behaviour:

```
# server
python3 -m server.monitor --interval 0.5 --output results/baseline-monitor.csv
# monitor (the "real user")
python3 -m client.sustained_http_load --url http://10.42.0.81/ \
    --duration 20 --timeout 2 --concurrency 10 --output results/baseline-http.csv
```

This produced `results/baseline-http.csv` (2302 requests, all successful — shown as the first command in Figure 5) and `results/baseline-monitor.csv`.

Step 2 — Shrink the queue, then launch the SYN flood

Before attacking, nginx's own `listen` backlog was reduced from its 511 default to a small value so that the attack has a realistic chance of overflowing it (see §2). On the attacker we then started the flood:

```
sudo python3 -m client.generator --continuous --rate 1000 --workers 8
```

Step 3 — Observe the victim under attack

While the flood ran, we watched half-open connections accumulate on the server and re-ran the legitimate-user probe from the monitor laptop:

```
# server: live count of half-open connections on port 80
watch -n0.5 'ss -Hnt state syn-recv "( sport = :80 )" | wc -l'
# server: packet-level proof SYNs are arriving
sudo tcpdump -ni <iface> 'tcp port 80 and tcp[tcpflags] & tcp-syn != 0 and tcp[
    tcpflags] & tcp-ack == 0'
# monitor: the "real user" probe during the attack window
python3 -m client.sustained_http_load --url http://10.42.0.81/ \
    --duration 20 --timeout 2 --concurrency 10 --output results/attack-http.csv
```

```

python3 baseline-monitor.csv sudo watch
00:13:21.084276 IP 10.42.0.1.41422 > 10.42.0.81.80: Flags [S], seq 287494418, win 16384, length 0
00:13:21.084425 IP 10.42.0.1.41540 > 10.42.0.81.80: Flags [S], seq 287494536, win 16384, length 0
00:13:21.084426 IP 10.42.0.1.41448 > 10.42.0.81.80: Flags [S], seq 287494444, win 16384, length 0
00:13:21.086204 IP 10.42.0.1.41507 > 10.42.0.81.80: Flags [S], seq 287494503, win 16384, length 0
00:13:21.086206 IP 10.42.0.1.41541 > 10.42.0.81.80: Flags [S], seq 287494537, win 16384, length 0
00:13:21.088054 IP 10.42.0.1.41495 > 10.42.0.81.80: Flags [S], seq 287494491, win 16384, length 0
00:13:21.090805 IP 10.42.0.1.41561 > 10.42.0.81.80: Flags [S], seq 287494557, win 16384, length 0
00:13:21.091057 IP 10.42.0.1.41538 > 10.42.0.81.80: Flags [S], seq 287494534, win 16384, length 0
00:13:21.091324 IP 10.42.0.1.41548 > 10.42.0.81.80: Flags [S], seq 287494544, win 16384, length 0
00:13:21.092473 IP 10.42.0.1.41430 > 10.42.0.81.80: Flags [S], seq 287494426, win 16384, length 0
00:13:21.092473 IP 10.42.0.1.41553 > 10.42.0.81.80: Flags [S], seq 287494549, win 16384, length 0
00:13:21.092474 IP 10.42.0.1.41456 > 10.42.0.81.80: Flags [S], seq 287494452, win 16384, length 0
00:13:21.094006 IP 10.42.0.1.41515 > 10.42.0.81.80: Flags [S], seq 287494511, win 16384, length 0
00:13:21.094017 IP 10.42.0.1.41549 > 10.42.0.81.80: Flags [S], seq 287494545, win 16384, length 0
00:13:21.096645 IP 10.42.0.1.41503 > 10.42.0.81.80: Flags [S], seq 287494499, win 16384, length 0
00:13:21.098941 IP 10.42.0.1.41569 > 10.42.0.81.80: Flags [S], seq 287494565, win 16384, length 0
00:13:21.099947 IP 10.42.0.1.41530 > 10.42.0.81.80: Flags [S], seq 287494526, win 16384, length 0
00:13:21.099948 IP 10.42.0.1.41546 > 10.42.0.81.80: Flags [S], seq 287494542, win 16384, length 0
00:13:21.099949 IP 10.42.0.1.41556 > 10.42.0.81.80: Flags [S], seq 287494552, win 16384, length 0
00:13:21.100838 IP 10.42.0.1.41438 > 10.42.0.81.80: Flags [S], seq 287494434, win 16384, length 0
00:13:21.100843 IP 10.42.0.1.41464 > 10.42.0.81.80: Flags [S], seq 287494460, win 16384, length 0
00:13:21.102265 IP 10.42.0.1.41523 > 10.42.0.81.80: Flags [S], seq 287494519, win 16384, length 0
00:13:21.102287 IP 10.42.0.1.41557 > 10.42.0.81.80: Flags [S], seq 287494553, win 16384, length 0
00:13:21.104312 IP 10.42.0.1.41511 > 10.42.0.81.80: Flags [S], seq 287494507, win 16384, length 0
00:13:21.107080 IP 10.42.0.1.41577 > 10.42.0.81.80: Flags [S], seq 287494573, win 16384, length 0
00:13:21.107769 IP 10.42.0.1.41554 > 10.42.0.81.80: Flags [S], seq 287494550, win 16384, length 0
00:13:21.107770 IP 10.42.0.1.41564 > 10.42.0.81.80: Flags [S], seq 287494560, win 16384, length 0
00:13:21.108469 IP 10.42.0.1.41446 > 10.42.0.81.80: Flags [S], seq 287494442, win 16384, length 0
00:13:21.109102 IP 10.42.0.1.41472 > 10.42.0.81.80: Flags [S], seq 287494468, win 16384, length 0
00:13:21.110403 IP 10.42.0.1.41565 > 10.42.0.81.80: Flags [S], seq 287494561, win 16384, length 0
00:13:21.110407 IP 10.42.0.1.41531 > 10.42.0.81.80: Flags [S], seq 287494527, win 16384, length 0

```

Figure 1: Victim: `tcpdump` showing a continuous stream of incoming SYN packets during the flood.

```

python3 baseline-monitor.csv sudo watch
Every 0.5s: ss -hnt state syn-recv "( sport = :80 )" | wc -l  jihad-Nitro-AN515-56: Sun Sep 27 00:15:05 2026
9

```

Figure 2: Victim: live `ss` count of SYN-RECEIVED connections on port 80 during the attack, pinned near the shrunk listen backlog.

Step 4 — Stop and collect evidence

We stopped the generator (`Ctrl+C`) and the monitor, then compared the baseline and attack CSVs.

4 (c) Observed Output on Each Machine

4.1 Victim PC (server)

On the victim the effect was visible two ways:

- `ss ...state syn-recv` — the half-open count rose from ≈ 0 at rest and stayed pinned near the shrunk listen backlog for the entire attack window (Figure 2), instead of draining the way it does at rest.
- `server/monitor.py` CSV — the `syn_recv_count` column tells the same story numerically: during the unprotected trial’s HTTP-probe window it sat at 9–12 continuously, whereas during the baseline and protected windows it fluctuated between 0 and 10 and regularly dropped back down (plotted in Figure 3). A pinned, non-draining queue — not merely a nonzero one — is what marks the unprotected trial as different.

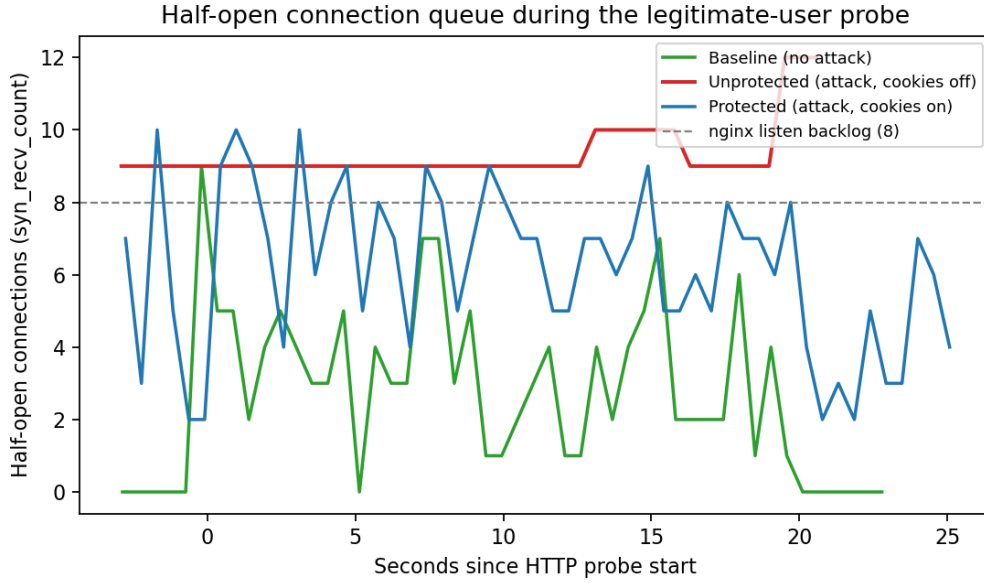


Figure 3: Victim: `syn_recv_count` over time from `server/monitor.py`, aligned to each trial’s HTTP-probe window. The unprotected trial (red) is pinned near the backlog limit for its whole duration; baseline (green) and protected (blue) both fluctuate freely, never getting stuck.

4.2 Monitor PC (the “real user” probe)

The legitimate-user probe run from the monitor laptop showed the service-level impact directly: comparing `baseline-http.csv`, `attack-http.csv`, and `protected-http.csv` (Table 2) is the ground truth for whether real users were affected.

```

results > attack-http.csv
51 2026-09-26T18:16:46.340174+00:00,50,10,False,2002.43,<curl error timed out>
52 2026-09-26T18:16:48.295989+00:00,51,10,False,2002.74,<curl error timed out>
53 2026-09-26T18:16:48.309436+00:00,52,10,False,2002.67,<curl error timed out>
54 2026-09-26T18:16:48.311459+00:00,53,10,False,2002.85,<curl error timed out>
55 2026-09-26T18:16:48.311844+00:00,54,10,False,2002.87,<curl error timed out>
56 2026-09-26T18:16:48.333263+00:00,55,10,False,2002.69,<curl error timed out>
57 2026-09-26T18:16:48.334406+00:00,56,10,False,2001.82,<curl error timed out>
58 2026-09-26T18:16:48.334753+00:00,57,10,False,2001.86,<curl error timed out>
59 2026-09-26T18:16:48.336634+00:00,58,10,False,2002.13,<curl error timed out>
60 2026-09-26T18:16:48.338642+00:00,59,10,False,2002.14,<curl error timed out>
61 2026-09-26T18:16:48.342691+00:00,60,10,False,2002.45,<curl error timed out>
62 2026-09-26T18:16:50.298785+00:00,61,10,False,2002.72,<curl error timed out>
63 2026-09-26T18:16:50.312238+00:00,62,10,False,2002.72,<curl error timed out>
64 2026-09-26T18:16:50.313516+00:00,63,10,False,2002.0,<curl error timed out>
65 2026-09-26T18:16:50.314449+00:00,64,10,False,2002.55,<curl error timed out>
66 2026-09-26T18:16:50.336053+00:00,65,10,False,2002.69,<curl error timed out>
67 2026-09-26T18:16:50.336526+00:00,66,10,False,2002.07,<curl error timed out>
68 2026-09-26T18:16:50.337249+00:00,67,10,False,2002.45,<curl error timed out>
69 2026-09-26T18:16:50.339182+00:00,68,10,False,2002.49,<curl error timed out>
70 2026-09-26T18:16:50.340437+00:00,69,10,False,2001.74,<curl error timed out>
71 2026-09-26T18:16:50.341707+00:00,70,10,False,2001.46,<curl error timed out>

arzon@ProBook:~/jihad_r_project/cyber_projects$ python3 -m client.sustained_http_load --url http://10.42.0.81/ \
--duration 20 --timeout 2 --concurrency 10 --output results/baseline-http.csv
arzon@ProBook:~/jihad_r_project/cyber_projects$ python3 -m client.sustained_http_load --url http://10.42.0.81/ \
--duration 20 --timeout 2 --concurrency 10 --output results/baseline-http.csv
saved 2302 HTTP results; successes: 2302/2302
arzon@ProBook:~/jihad_r_project/cyber_projects$ python3 -m client.sustained_http_load --url http://10.42.0.81/ \
--duration 20 --timeout 2 --concurrency 10 --output results/attack-http.csv
saved 100 HTTP results; successes: 0/100
arzon@ProBook:~/jihad_r_project/cyber_projects$

```

Figure 4: Monitor: the legitimate-user probe run during the unprotected flood, reporting 0/100 successes, every request timing out at the 2s limit.

```

results > protected-http.csv
8110 2026-09-26T18:22:38.565965+00:00,8109,10,True,31.47,200,
8111 2026-09-26T18:22:38.566763+00:00,8110,10,True,31.43,200,
8112 2026-09-26T18:22:38.567205+00:00,8111,10,True,32.32,200,
8113 2026-09-26T18:22:38.629643+00:00,8112,10,True,227.15,200,
8114 2026-09-26T18:22:38.637646+00:00,8113,10,True,233.83,200,
8115 2026-09-26T18:22:38.785622+00:00,8114,10,True,251.4,200,
8116 2026-09-26T18:22:39.178251+00:00,8115,10,True,1070.72,200,
8117 2026-09-26T18:22:39.291041+00:00,8116,10,True,1012.06,200,
8118 2026-09-26T18:22:39.614139+00:00,8117,10,True,1059.35,200,
8119 2026-09-26T18:22:40.858217+00:00,8118,10,False,2302.63,,[Errno 104] Connection reset by peer
8120

arzon@ProBook:~/jihad_r_project/cyber_project$ python3 -m client.sustained_http_load --url http://10.42.0.81/ \
--duration 20 --timeout 2 --concurrency 10 --output results/baseline-http.csv
saved 2302 HTTP results; successes: 2302/2302
arzon@ProBook:~/jihad_r_project/cyber_project$ python3 -m client.sustained_http_load --url http://10.42.0.81/ \
--duration 20 --timeout 2 --concurrency 10 --output results/attack-http.csv
saved 100 HTTP results; successes: 0/100
arzon@ProBook:~/jihad_r_project/cyber_project$ python3 -m client.sustained_http_load --url http://10.42.0.81/ \
--duration 20 --timeout 2 --concurrency 10 --output results/protected-http.csv
saved 8118 HTTP results; successes: 8117/8118
arzon@ProBook:~/jihad_r_project/cyber_project$

```

Figure 5: Monitor terminal: the three `sustained_http_load` invocations back to back — baseline (2302/2302 succeeded), unprotected attack (0/100 succeeded), and protected attack (8117/8118 succeeded).

Table 2: Legitimate HTTP probe: baseline vs. unprotected attack vs. protected attack.

Condition	Success rate	Median latency	Timeouts
Baseline (no attack)	2302 / 2302 (100.00%)	48.5 ms	0
Unprotected (attack)	0 / 100 (0.00%)	2002.4 ms	100
Protected: SYN cookies	8117 / 8118 (99.99%)	16.9 ms	0

Note: the one failure in the protected trial was a single **Connection reset by peer**, not a timeout — consistent with an essentially fully-restored service under active flood.

5 (b) Was the Attack Successful?

We treat the attack as **successful** only if two things hold together, so that mere packet generation is not mistaken for impact:

1. **The victim held the attack state:** `syn_recv_count` spiked and stayed pinned near the (shrunk) listen backlog for the entire attack window, instead of draining the way it does at rest; and
2. **There was a measurable service effect:** the legitimate probe’s success rate collapsed from 100% to 0%, with every request timing out at the full 2 s limit (Table 2).

Both held in our unprotected trial, so we call the attack **successful**.

Why it succeeded. The half-open queue is a finite kernel resource, sized in our lab to a small backlog (§2). Because the flood’s SYNs never complete the handshake, each one pins a backlog slot until it is retransmitted away; a fast enough, sustained arrival rate keeps the queue continuously full, so genuine handshakes lose the race for a free slot. This matches Figure 3 directly: the unprotected line never drops the way baseline and protected do.

Factors that could have made the attack fail, and what we checked.

- **Wi-Fi / hotspot rate limits:** a single raw-socket process tops out around 800–1000 packets/second regardless of the requested rate, which is why the generator is run with `-workers 8` to reach a genuine flood rate.
- **Access-point filtering:** some hotspots silently drop packets whose source IP doesn't match the sending client, before they ever reach the victim. We checked this directly with `tcpdump` on the victim during the flood (Figure 1) and confirmed packets were arriving, so this was not the limiting factor here.
- **An oversized listen backlog:** our first attempt used nginx's default backlog of 511, and no effect was observed at all — not because the attack failed to land, but because 511 free slots plus sub-millisecond same-LAN handshakes meant real requests essentially never lost the race for a slot. This was diagnosed and fixed as described in §2, and is the main methodological lesson of this lab.
- **Kernel SYN cookies:** if enabled, the victim answers backlog pressure with cookies instead of dropping SYNs, and keeps serving — the attack still “lands” but does not deny service. This is exactly the protected case in §6, not a failure of the attack.

6 (d) Countermeasures

6.1 Countermeasure — stateless SYN cookies (evaluated)

The primary defense removes the resource the attack targets instead of trying to identify attackers. When enabled (`net.ipv4.tcp_syncookies=1`), the kernel stops storing per-connection state once the backlog is under pressure and instead encodes the connection's identity, a coarse timestamp, and the client's MSS into the 32-bit initial sequence number of the **SYN-ACK** (the “cookie”). Only a client that truly received the **SYN-ACK** can echo `cookie+1` in its **ACK**, so the kernel recomputes and verifies the cookie and rebuilds the connection — while a spoofed **SYN** costs one **SYN-ACK** and zero memory. The project also includes a standalone reference implementation of this exact mechanism, `server/syn_cookies.py` (`python3 -m server.syn_cookies`), for inspecting the encode/verify steps in isolation; it was not re-run for this report since the kernel's own cookies are what actually protect nginx.

```
# on the server, before the protected trial:
sudo sysctl -w net.ipv4.tcp_syncookies=1
```

This is exactly the *protected* trial already presented in §7: under the identical flood that drove the unprotected trial to 0% success, enabling cookies restored the legitimate probe to 99.99% success (Table 2), and the queue-occupancy plot (Figure 3) shows it fluctuating rather than pinned, just like the undisturbed baseline. SYN cookies never block anyone — they don't decide who is an attacker, they simply refuse to keep state until a handshake is proven complete — so a legitimate client's handshake succeeds normally with no collateral damage. That is precisely why this, rather than per-source blocking, is the primary real-world defense.

6.2 Countermeasure — `server/defense.py` (SYN Guard): design only, evaluation incomplete

A second, application-level defense was designed but **not empirically evaluated in time for this report**; the discussion below is a design analysis of the code as written, not a report of measured results, and no block-event data or table row accompanies it.

`defense.py` runs on the victim on a timer. Each tick it reads the half-open connections from `ss`, counts them per source IP, and for any source whose half-open count meets a threshold it inserts a temporary `iptables` rule dropping further SYNs from that address, logging every action to a CSV and lifting the block after a cooldown:

```
sudo python3 -m server.defense --interval 0.5 --threshold 5 \
  --block-seconds 30 --output results/defense-events.csv
```

Expected limitation — it can also block legitimate users. `defense.py` decides guilt purely by *source IP address*, so its mitigation is, by design, inseparable from collateral damage:

- **It blocks innocent addresses.** A SYN flood uses *spoofed* source IPs. When `defense.py` blocks a flagged address, it is blocking an address the attacker merely *claimed* to be — which may belong to a real, innocent host (or the whole subnet behind a NAT). That legitimate user is now firewalled off even though they did nothing.
- **It cannot cope with a widely distributed/spoofed flood.** If each fake source stays just under the threshold, or if thousands of distinct spoofed IPs are used, the defense either never triggers or must block huge ranges of addresses — again sweeping up legitimate clients.
- **Threshold is a blunt instrument.** Set it low and you block bursty but legitimate clients; set it high and a real flood slips under it. There is no setting that cleanly separates attacker from victim, because the signal it uses (source IP) is exactly the field the attacker forges.

In short, `defense.py` is designed as an observable, tunable detector against a *few real, high-volume* sources, but by construction it can punish legitimate users, so it is not intended as a safe standalone defense against a spoofed SYN flood. Confirming this experimentally (including whether the monitor’s own legitimate traffic gets blocked under load) is left as follow-up work.

6.3 Comparison

Table 3: The two countermeasures side by side. Only the SYN-cookies row is backed by measurement in this report.

	<code>syn_cookies.py</code> / <code>kernel</code>	<code>defense.py</code> (SYN Guard)
Status in this report	evaluated (Table 2)	design only, not evaluated
Strategy	store no half-open state	detect & block noisy source IPs
State kept	none	one timer per blocked IP
Blocks legit users?	No	Expected: yes (side effect)
Best against	spoofed-source floods	a few real, high-volume sources
Kernel analogue	<code>net.ipv4.tcp_syncookies</code>	none (application level)

7 Results Summary

Across the three measured trials — **baseline**, **unprotected** (attack, cookies off), and **protected** (attack, cookies on) — we compare the victim’s `syn_recv_count` and the legitimate probe’s success/latency (Table 2, Figure 3). The story is exactly what the theory predicts: an oversized queue (511, our first attempt) hides the attack entirely; once the queue is genuinely small, the flood pins it full and HTTP collapses to 0% success when undefended; enabling SYN cookies under the identical flood restores HTTP to 99.99% success while the queue-occupancy trace stops being pinned. `defense.py`’s per-source blocking approach is analyzed in §6.2 but was not run, so no claim is made about its measured effect here.

8 Conclusion

We reproduced a TCP SYN-flood attack end to end on our own isolated hotspot network and measured its effect on both the kernel's half-open queue and real HTTP service. Our first attempt showed no effect at all, and diagnosing why — nginx's own listen backlog, not the kernel sysctl, is what actually governs the queue, and 511 free slots is enough to hide the attack entirely on a fast LAN — was as much a part of this lab as the attack itself (§2). Once the queue was genuinely shrunk, the flood drove a real, legitimate user's success rate from 100% to 0%. Enabling stateless SYN cookies (`net.ipv4.tcp_syncookies`) under the identical flood restored success to 99.99%, without ever deciding who to block — which is why it is the primary real-world countermeasure. A second, application-level countermeasure (`server/defense.py`) was designed and analyzed for its expected trade-off (it can block legitimate users because it acts on the spoofable source-IP field), but its empirical evaluation remains incomplete and is left as follow-up work.

A Reproduction Commands

```
# --- Server / victim (10.42.0.81) ---
# one-time: shrink nginx's own listen backlog (see Sec. "Shrinking the queue")
# edit /etc/nginx/sites-available/default: listen 80 ... backlog=8;
# sudo nginx -t && sudo systemctl restart nginx
# ss -ltn 'sport = :80' # confirm Send-Q reads 8

python3 -m server.monitor --interval 0.5 --output results/attack-monitor.csv
watch -n0.5 'ss -Hnt state syn-recv "( sport = :80 )" | wc -l'
sudo tcpdump -ni <iface> 'tcp port 80 and tcp[tcpflags] & tcp-syn != 0'

# --- Attacker (separate laptop) ---
sudo python3 -m client.generator --continuous --rate 1000 --workers 8

# --- Monitor / legitimate user probe (separate laptop) ---
python3 -m client.sustained_http_load --url http://10.42.0.81/ \
    --duration 20 --timeout 2 --concurrency 10 --output results/attack-http.csv

# --- Countermeasure: SYN cookies (evaluated) ---
sudo sysctl -w net.ipv4.tcp_syncookies=1 # protect nginx
python3 -m server.syn_cookies --connections 1000 # standalone demo of the mechanism

# --- Countermeasure: per-source blocker (designed, not yet evaluated) ---
sudo python3 -m server.defense --interval 0.5 --threshold 5 \
    --block-seconds 30 --output results/defense-events.csv
```